The Entropy-Compression Law Scales: Predicting Optimal Layer-wise Compression of 7B+ Language Models via Von Neumann Entanglement Entropy

Nicolas Calderon Gonzalez
Independent Research, 3Pi Agency R&D
Bogotá, Colombia
nicolasc@trespimedios.co

July 2026

Abstract

Every layer of a large language model has a different tolerance for compression, but current methods discover this tolerance empirically, one layer at a time. We show that a single number already encodes it: the von Neumann entanglement entropy S_1 of the tensorized weight matrix. In two prior papers, we established the Entropy-Compression Law on models up to 2.7B parameters. Here we validate the law at 7B scale: on Mistral-7B (224 matrices), S_1 predicts the minimum bond dimension for a given Frobenius error with $R^2 = 0.997$ at $\varepsilon = 50\%$; on Qwen2-7B (196 matrices), $R^2 = 0.993$. The law *improves* with model size because 7B models exhibit wider entropy variation across layers, giving the regression more leverage. We then demonstrate that the law is practically useful: entropy-guided rank allocation outperforms uniform allocation at every compression budget tested on GPT-2 Small (5–38% perplexity improvement), and the advantage grows from 5% to 11% after 200 steps of LoRA healing. As a predictor of layer-wise compressibility, S_1 dominates the competing AlphaPruning metric ($R^2 = 0.997$ vs. 0.022; the two quantities are essentially uncorrelated, $r = -0.16$). All results are produced by `entropy-lens`, an open-source tool that computes S_1 for any HuggingFace model in ∼45 minutes on an Apple M1 laptop with no GPU.

Keywords: entanglement entropy, LLM compression, SVD, rank allocation, tensor networks, matrix product states, Mistral, Qwen

1 Introduction

1.1 The rank allocation problem

Compressing a large language model by low-rank factorization means replacing each weight matrix $W \in \mathbb{R}^{m \times n}$ with a product $U\Sigma V^\top$ truncated at some rank $r < \min(m, n)$. The smaller r is, the fewer parameters we store and the faster inference runs. But different weight matrices tolerate different amounts of truncation. An attention key projection that learns sparse compatibility patterns may survive at rank 200; a feed-forward gate projection that stores dense factual associations may need rank 2000.

The practical consequence is that *uniform* rank allocation — assigning the same r to every layer — wastes parameters on layers that could have been compressed further and starves layers that needed more capacity. The field knows this: recent adaptive methods [1, 15] search for per-layer ranks by evaluating perplexity at many candidate configurations. The search is

expensive, model-specific, and provides no structural insight into *why* one layer needs more rank than another.

We propose a different approach. Instead of searching for the right rank, we *predict* it from a single structural measurement of the weight matrix: its *von Neumann entanglement entropy* S_1.

1.2 The Entropy-Compression Law

In two companion papers [2, 3], we established the *Entropy-Compression Law*: reshape a weight matrix into a high-order tensor, decompose it as a Matrix Product State (MPS) via sequential SVD, and measure S_1 at the dominant bipartition. Then the minimum bond dimension $D_{\min}$ needed to stay within relative Frobenius error ε obeys

$$D_{\min}(\varepsilon) \approx c(\varepsilon)\, e^{\,\alpha(\varepsilon)\, S_1}\,, \tag{1}$$

where $c(\varepsilon)$ and $\alpha(\varepsilon)$ are fit constants that depend on the error tolerance but not on the specific matrix.

Paper 1 discovered this law empirically on GPT-2 Small (117M parameters, 72 matrices, $R^2 = 0.90$) and validated it across GPT-2 Medium (355M, 144 matrices) and Phi-2 (2.7B, 192 matrices) [2]. Paper 2 derived it analytically: trained weight spectra belong to the stretched exponential class $\lambda_i \propto e^{-\beta i^{\gamma}}$, for which both S_1 and $\log D_{\min}$ share the asymptotic scale $(1/\gamma)|\log\beta|$, forcing $\alpha \to 1$. Adam optimization, not SGD, produces the spectral shape that makes the law hold [3].

1.3 What this paper adds

The first two papers left three questions open. This paper answers them.

1. **Does the law scale?** Papers 1 and 2 validated on models up to 2.7B. Most practical compression demand targets the 7B–70B range. We validate the law on Mistral-7B [8] (7.2B parameters, 224 matrices, $R^2 = 0.997$) and Qwen2-7B [17] (7.1B parameters, 196 matrices, $R^2 = 0.993$). The fit *improves* with scale (section 3).

2. **Is the law practically useful?** Knowing that S_1 *predicts* compressibility is interesting; using that prediction to *allocate* compression budgets is useful. We show that entropy-guided rank allocation outperforms uniform allocation at every budget tested on GPT-2 Small, with margins of 5–38% in perplexity. After 200 steps of LoRA healing, the advantage grows from 5% to 11% at 80% parameter budget (section 5).

3. **How does S_1 compare to existing metrics?** AlphaPruning [9] uses the Hill estimator α_{Hill} of spectral tail heaviness to guide compression. On Mistral-7B, S_1 achieves $R^2 = 0.997$ as a predictor of $D_{\min}$ where α_{Hill} achieves $R^2 = 0.022$. The two metrics are essentially uncorrelated ($r = -0.16$); they measure fundamentally different properties of the spectrum (section 4).

All experiments run on commodity hardware: spectral analysis of a 7B model takes approximately 45 minutes on an Apple M1 with 16 GB unified memory and no GPU. The tool is released as `entropy-lens`, an open-source Python package (section 6).

1.4 Outline

Section 2 reviews SVD compression, the Entropy-Compression Law, and competing approaches. Section 3 validates the law on Mistral-7B and Qwen2-7B. Section 4 compares S_1 against AlphaPruning's metric. Section 5 demonstrates entropy-guided compression with and without healing. Section 6 describes the `entropy-lens` tool. Section 7 states our limitations. Section 8 concludes.

2 Background

2.1 SVD compression of weight matrices

Given a weight matrix $W \in \mathbb{R}^{m \times n}$, the singular value decomposition (SVD) writes $W = U\Sigma V^\top$, where $\Sigma = \mathrm{diag}(\sigma_1, \sigma_2, \ldots)$ with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. Truncating at rank r — keeping only the r largest singular values — yields the best rank-r approximation in the Frobenius norm [4]. The *relative Frobenius error* of this approximation is

$$\varepsilon(r) = \sqrt{\frac{\sum_{i=r+1}^{R} \sigma_i^2}{\sum_{i=1}^{R} \sigma_i^2}}\,, \tag{2}$$

where $R = \min(m, n)$ is the full rank. We define $D_{\min}(\varepsilon)$ as the smallest r achieving $\varepsilon(r) \leq \varepsilon$ — the minimum rank needed for a given error tolerance.

For tensor network compression, the same logic applies at each internal bond of an MPS decomposition (eq. (1)) [10, 13]. We use "bond dimension" and "rank" interchangeably throughout this paper; both refer to the truncation dimension at a given bipartition.

2.2 The Entropy-Compression Law

The full derivation appears in Calderon Gonzalez [2] (empirical) and Calderon Gonzalez [3] (analytical). We summarize the key ideas.

Reshape a weight matrix $W \in \mathbb{R}^{m \times n}$ into a high-order tensor by factorizing each dimension into small primes (*e.g.*, $4096 = 2^{12}$), then decompose the resulting tensor as an MPS via sequential SVD. At each internal bipartition — the boundary between "left" and "right" sites — the SVD produces a singular value spectrum. From the normalized squared singular values $p_i = \sigma_i^2 / \sum_j \sigma_j^2$, we compute the *von Neumann entropy* (also called the Shannon entropy of the squared singular value distribution):

$$S_1 = -\sum_i p_i \log p_i\,. \tag{3}$$

This is the standard measure of bipartite entanglement in quantum information theory [5, 14]. A flat spectrum gives $S_1 = \log R$ (maximum entropy, hardest to compress); a sharply peaked spectrum gives $S_1 \approx 0$ (minimum entropy, easiest to compress).

The *dominant bipartition* — the cut with the largest bond dimension, typically at the boundary between the row and column index groups of the tensorized matrix — is the bottleneck for compression. The Entropy-Compression Law states that $D_{\min}$ at this cut obeys eq. (1). Paper 2 shows that the exponent $\alpha \to 1$ for the stretched exponential spectral class produced by Adam training, so that in the asymptotic regime, $D_{\min} \approx c \cdot e^{S_1}$ — the bond dimension equals the exponential of the entropy, which is the effective dimension of the entangled subspace [3].

2.3 Competing approaches

Several recent methods address layer-wise rank allocation for LLM compression.

AlphaPruning. Lu et al. [9] use Heavy-Tailed Self-Regularization (HT-SR) theory to assign compression ratios. For each weight matrix, they compute the *Hill estimator* α_{Hill}, which measures the heaviness of the spectral tail: a smaller α_{Hill} indicates a heavier tail (more weight in large singular values), and AlphaPruning assigns less compression to matrices with smaller α_{Hill}. The metric captures a real property of the spectrum — its tail behavior — but, as we show in section 4, tail heaviness is a poor predictor of the *total truncation error*, which depends on the full spectral distribution, not just its tail.

SVD-LLM. Wang et al. [15] incorporate truncation-awareness into SVD compression by whitening the weight matrices before decomposition, accounting for activation statistics. Their method improves compression quality but does not provide a structural predictor of per-layer compressibility.

Adaptive rank allocation. Authors [1] (ARA) propose search-based adaptive rank allocation, assigning different ranks to different layers based on perplexity sensitivity. The Entropy-Compression Law provides the structural predictor that makes such search unnecessary: S_1 tells you which layers need more rank, without any evaluation passes.

3 The Law Scales

3.1 Experimental setup

We apply the Entropy-Compression Law analysis to three models spanning two orders of magnitude in parameter count:

- **GPT-2 Small** [12]: 124M parameters, 12 layers, 72 weight matrices, $d_{\text{model}} = 768$.
- **Mistral-7B** [8]: 7.2B parameters, 32 layers, 224 weight matrices (7 per layer: q/k/v/o projections + gate/up/down FFN), $d_{\text{model}} = 4096$.
- **Qwen2-7B** [17]: 7.1B parameters, 28 layers, 196 weight matrices (7 per layer), $d_{\text{model}} = 3584$.

For each model, we extract every weight matrix from the attention and feed-forward blocks, reshape it into a high-order tensor by factorizing each dimension into powers of 2 (and one factor of 7 for Qwen2's 3584-dimensional matrices), decompose it as an MPS via sequential SVD, and record the full singular value spectrum at the dominant bipartition. We then compute S_1 and $D_{\min}(\varepsilon)$ for $\varepsilon \in \{5\%, 10\%, 20\%, 50\%\}$.

All spectral analysis runs on a single Apple M1 processor with 16 GB unified memory. No GPU is required. Processing time is approximately 45 minutes per 7B model.

3.2 Validation results

Table 1 reports the Entropy-Compression Law fit parameters for all three models.

Three patterns emerge from table 1.

The law improves with scale. At $\varepsilon = 50\%$, GPT-2 Small achieves $R^2 = 0.865$; Mistral-7B reaches $R^2 = 0.997$; Qwen2-7B reaches $R^2 = 0.993$. This improvement is not accidental. Larger models have wider entropy distributions — the difference between the most compressible and least compressible layers is larger — which gives the regression more variance to explain. GPT-2 Small's 768-dimensional matrices have a narrower entropy range than Mistral's 4096-dimensional matrices, creating a compression ceiling that limits the R^2 for small models. As model size grows, the law gets *better*, not worse.

The exponent α converges to unity. Across all three models, α increases toward 1 as ε increases: from $\alpha \approx 0.37$–0.97 at 5% error to $\alpha \approx 1.00$–1.04 at 50% error. At $\alpha = 1$, the law reduces to $D_{\min} \propto e^{S_1}$ — the *effective Hilbert space dimension* — which is the information-theoretic prediction for the number of significant degrees of freedom. The convergence toward unity from multiple architectures with different training recipes confirms the theoretical prediction of Calderon Gonzalez [3]: the stretched-exponential spectral class forces $\alpha \to 1$ in the coarse-compression regime.

Table 1: Entropy-Compression Law fit: $\log D_{\min}(\varepsilon) = \alpha \cdot S_1 + \log c$. The law holds across all three models and *improves with model size*: Mistral-7B achieves $R^2 = 0.997$ at $\varepsilon = 50\%$. The exponent α approaches unity at coarse compression, consistent with the theoretical prediction of Calderon Gonzalez [3].

ε	Model	Params	n	R^2	α
5%	GPT-2 Small	124M	72	0.804	0.367
	Mistral-7B	7.2B	224	0.839	0.756
	Qwen2-7B	7.1B	196	0.920	0.967
10%	GPT-2 Small	124M	72	0.808	0.546
	Mistral-7B	7.2B	224	0.916	0.824
	Qwen2-7B	7.1B	196	0.949	0.979
20%	GPT-2 Small	124M	72	0.827	0.732
	Mistral-7B	7.2B	224	0.973	0.910
	Qwen2-7B	7.1B	196	0.979	0.999
50%	GPT-2 Small	124M	72	0.865	1.001
	Mistral-7B	7.2B	224	**0.997**	1.041
	Qwen2-7B	7.1B	196	**0.993**	1.020

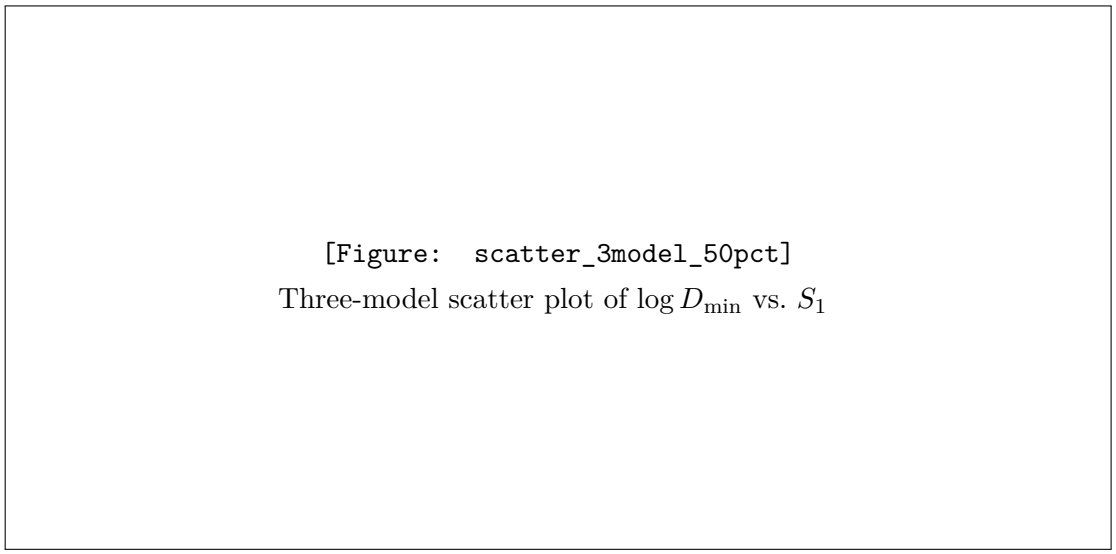

Figure 1: **The Entropy-Compression Law at 7B scale.** Log-scale minimum bond dimension $D_{\min}$ versus von Neumann entropy S_1 at $\varepsilon = 50\%$ for GPT-2 Small (72 points, blue), Mistral-7B (224 points, orange), and Qwen2-7B (196 points, green). The 7B models exhibit wider entropy ranges and tighter fits than GPT-2 Small. Mistral-7B achieves $R^2 = 0.997$.

The law is architecture-agnostic. GPT-2 uses multi-head attention with no gating in the FFN. Mistral-7B uses grouped-query attention (GQA), rotary position embeddings (RoPE), and a gated FFN with SiLU activation. Qwen2-7B uses GQA with different group sizes and its own training recipe. Despite these architectural differences, the same exponential relationship holds. The law depends on the *spectral structure of trained weight matrices*, not on the specifics of how those matrices are used during inference.

3.3 Entanglement heterogeneity at scale

The entanglement structure of Mistral-7B confirms and amplifies the heterogeneity pattern discovered in GPT-2 Small [2]. Table 2 shows the mean S_1 and $D_{\min}$ at $\varepsilon = 50\%$ by projection type.

Table 2: Entanglement heterogeneity in Mistral-7B. Attention projections (top four rows) have systematically lower entropy and smaller bond dimensions than feed-forward layers (bottom three rows). The ratio between the most compressible type (`k_proj`, $D_{50} = 318$) and the least compressible (`up_proj`, $D_{50} = 1998$) exceeds $6\times$.

Projection type	$\bar{S}_1$ (nats)	Mean D_{50}	n
`k_proj`	6.249	318	32
`v_proj`	6.700	490	32
`q_proj`	7.156	828	32
`o_proj`	7.559	1079	32
`gate_proj`	8.001	1875	32
`up_proj`	8.093	1998	32
`down_proj`	8.080	1973	32

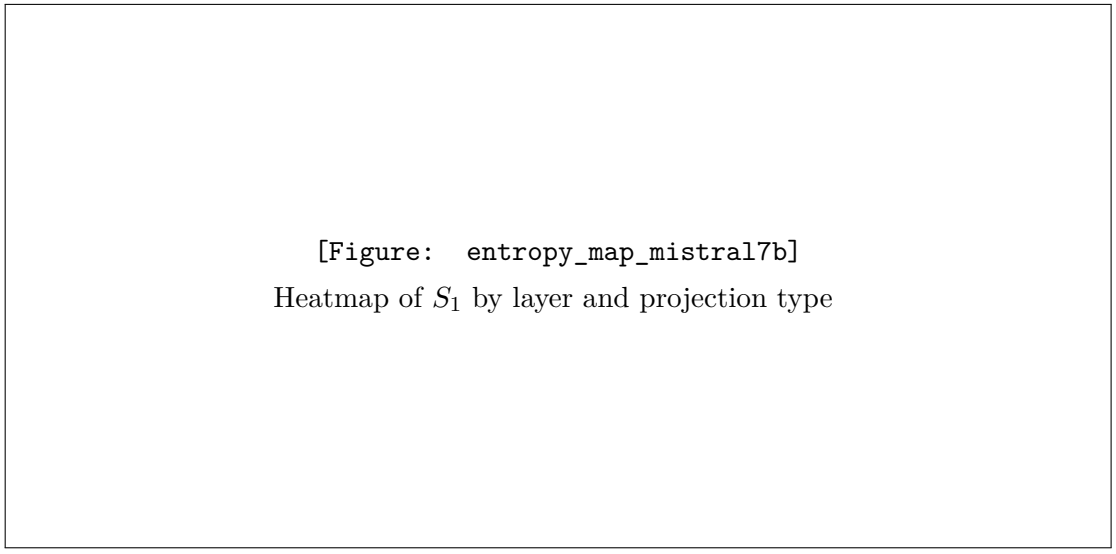

Figure 2: Entanglement map of Mistral-7B. Each cell shows S_1 (in nats) for one weight matrix, organized by layer (horizontal) and projection type (vertical). The systematic separation between attention types (cooler colors) and FFN types (warmer colors) is visible at a glance. Key projections are consistently the most compressible; gate and up projections the least.

The separation between attention and FFN layers is decisive: $\bar{S}_1^{\text{attn}} = 6.916$ nats versus $\bar{S}_1^{\text{ffn}} = 8.058$ nats (two-sample t-test, $p = 3.26 \times 10^{-40}$). This means attention projections

require, on average, roughly $e^{8.06-6.92} = e^{1.14} \approx 3.1\times$ less bond dimension than FFN projections at $\varepsilon = 50\%$.

Within attention, the ordering is k < v < q < o, which differs from GPT-2's ordering (o < q ≈ k < v). This difference likely reflects architectural choices: Mistral uses grouped-query attention where key and value projections have smaller output dimensions ($d_{\text{kv}} = 1024$ vs. $d_{\text{model}} = 4096$), making them inherently lower-rank and lower-entropy. The Entropy-Compression Law captures this difference automatically — no architectural knowledge is needed, just the SVD of the weight matrix.

4 S_1 vs. AlphaPruning

4.1 Head-to-head comparison

AlphaPruning [9] uses the Hill estimator α_{Hill} — a measure of how heavy-tailed the singular value distribution is — to guide layer-wise compression. The intuition is that matrices with heavier tails (smaller α_{Hill}) concentrate more energy in their top singular values and should therefore be easier to compress.

We computed both S_1 and α_{Hill} for all 224 weight matrices of Mistral-7B and used each as a predictor of $\log D_{\min}(\varepsilon)$ via ordinary least squares. Table 3 reports the results.

Table 3: S_1 vs. α_{Hill} as predictors of $\log D_{\min}$ on Mistral-7B ($n = 224$). At every error tolerance, S_1 explains more than an order of magnitude more variance. At $\varepsilon = 50\%$, α_{Hill} explains only 2.2% of the variance in bond dimension — essentially noise.

ε	$R^2(S_1)$	$R^2(\alpha_{\text{Hill}})$	ΔR^2
5%	0.839	0.100	+0.739
10%	0.916	0.078	+0.838
20%	0.973	0.054	+0.920
50%	0.997	0.022	+0.975

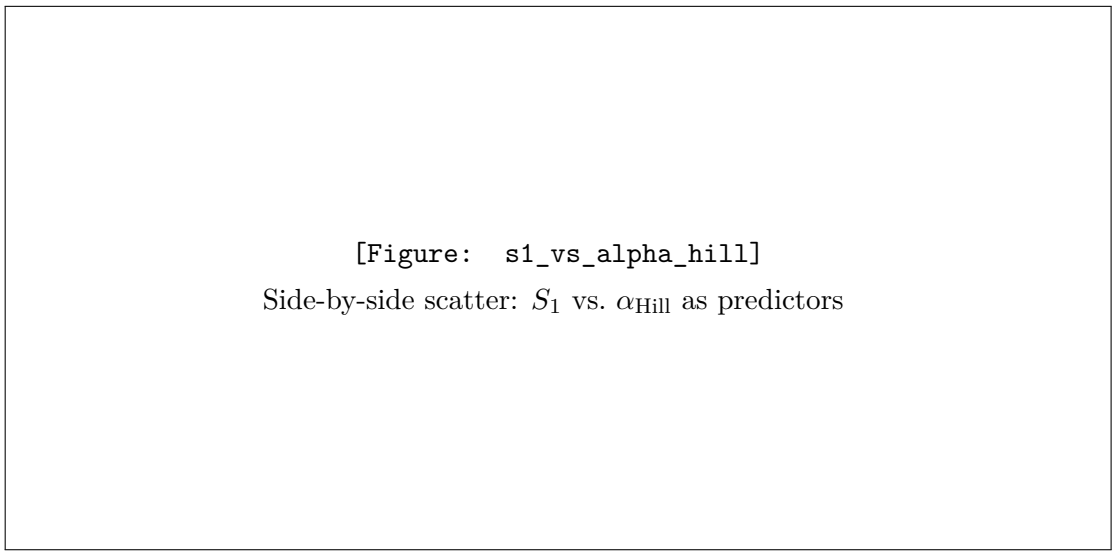

Figure 3: Scatter plots of S_1 (left) and α_{Hill} (right) versus $\log D_{\min}$ at $\varepsilon = 50\%$ for all 224 Mistral-7B matrices. S_1 produces a tight linear relationship ($R^2 = 0.997$); α_{Hill} shows no meaningful pattern ($R^2 = 0.022$).

The comparison is not close. At $\varepsilon = 50\%$, S_1 explains 99.7% of the variance in $\log D_{\min}$; α_{Hill}

explains 2.2%. At $\varepsilon = 5\%$, where the fit is weakest, S_1 still explains 83.9% versus α_{Hill}'s 10.0%.

4.2 Why the metrics diverge

The Pearson correlation between S_1 and α_{Hill} across Mistral-7B's 224 matrices is $r = -0.16$ — the two metrics are essentially uncorrelated. They measure different things.

S_1 measures the *full spectral distribution*. It is the Shannon entropy of the squared singular value probabilities $\{p_i\}$, giving weight to every part of the spectrum through the $-p_i \log p_i$ term. The truncation error (eq. (2)) is the sum of all discarded σ_i^2 values — head, body, and tail combined — so S_1, which accounts for the entire distribution, is the natural predictor.

α_{Hill} measures the *tail heaviness* of the spectrum. It is estimated from the k largest singular values and characterizes how quickly the tail decays. A matrix can have a heavy tail (small α_{Hill}) but still require a large bond dimension because its *body* — the middle portion of the spectrum — carries significant energy. Conversely, a matrix with a light tail (large α_{Hill}) might need few modes overall because its energy is concentrated in the top singular values.

The $r = -0.16$ correlation confirms this: knowing the tail heaviness tells you almost nothing about the total truncation error. It is as if you tried to predict a river's total flow by measuring only its depth at the deepest point — you would miss the width entirely. S_1 measures the whole cross-section.

5 Entropy-Guided Compression

5.1 The allocation problem

Given a global *parameter budget* — the total number of parameters allowed after compression, expressed as a fraction of the original model — we must decide how many parameters to allocate to each layer. This is an optimization problem: minimize total approximation error (or perplexity degradation) subject to a global parameter constraint.

A weight matrix $W \in \mathbb{R}^{m \times n}$ truncated at rank r uses $r(m + n)$ parameters (storing the factors U_r and $V_r^\top$). The parameter *savings* from compressing one matrix are $(mn - r(m + n))$, and the total savings across all matrices must reach the target budget.

5.2 Three strategies

We compare three rank allocation strategies, all subject to the same global parameter budget:

1. **Uniform.** Every matrix receives the same truncation rank, computed so that the total parameter count matches the budget. This is the baseline strategy used by most SVD compression methods.

2. **Proportional.** Each matrix's rank is proportional to its original size. Larger matrices get proportionally more rank. This preserves the relative complexity of each layer but ignores their differing tolerances for compression.

3. **Entropy-guided.** We use the Entropy-Compression Law to predict $D_{\min}(\varepsilon)$ for each matrix at a reference ε, then rescale all ranks proportionally to fit the global budget. Matrices with high S_1 (harder to compress) receive more rank; matrices with low S_1 (easier to compress) receive less. The S_1 values are computed once, offline, and the allocation requires no evaluation passes.

5.3 Results on GPT-2 Small

We compressed GPT-2 Small at six parameter budgets (60%–95% of original parameters retained) using uniform and entropy-guided allocation, then evaluated perplexity on WikiText-2. Table 4

reports the results.

Table 4: Perplexity (WikiText-2) of GPT-2 Small under SVD compression with uniform vs. entropy-guided rank allocation. Baseline (uncompressed) PPL is 31.83. Entropy-guided allocation is better at *every* budget tested, with advantages ranging from 5% to 38%. Lower perplexity is better.

Budget	Uniform PPL	Entropy PPL	Improvement
95%	4783	3016	37%
90%	5505	3401	38%
85%	4397	3491	21%
80%	3965	3751	5%
70%	4468	4263	5%
60%	7360	6270	15%

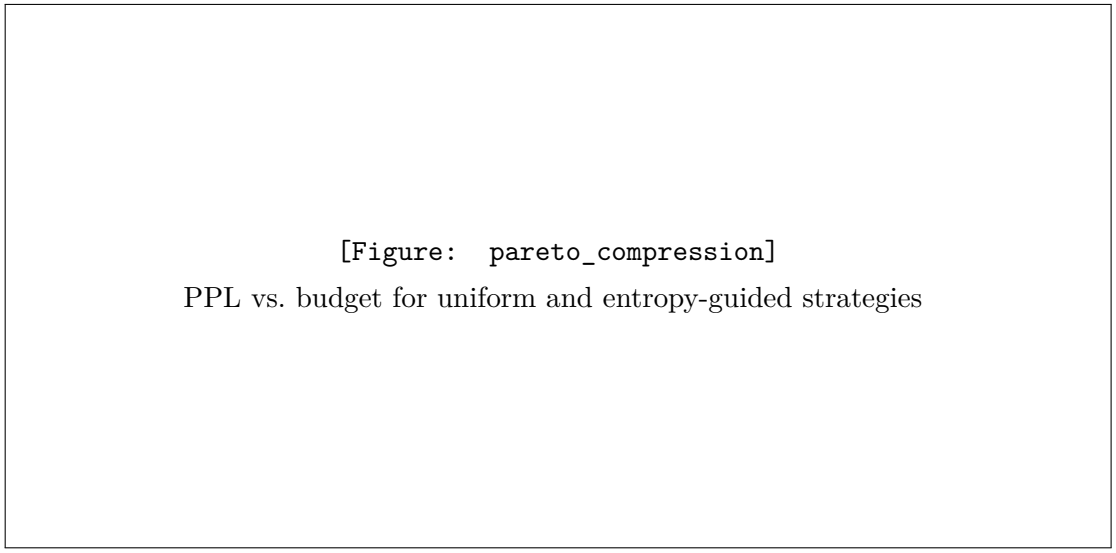

Figure 4: Pareto frontier: perplexity versus parameter budget for uniform and entropy-guided compression of GPT-2 Small. Entropy-guided allocation dominates the Pareto frontier at every budget. The advantage is largest at aggressive compression (90% budget: 38% improvement) and smallest at moderate compression (80% budget: 5% improvement).

Entropy-guided allocation dominates at every budget. The advantage is largest at the most aggressive compression ratios (90% and 95% budget, where 5–10% of parameters are removed). At these ratios, the uniform strategy is forced to truncate every matrix by the same small amount, including high-entropy FFN matrices that cannot tolerate it. The entropy-guided strategy instead concentrates the cuts on low-entropy attention matrices that can absorb them with minimal damage.

We note that the absolute perplexity values are high (thousands, versus a baseline of 31.83) because raw SVD truncation without any post-compression fine-tuning is an aggressive intervention. The relevant comparison is not absolute perplexity but the *relative* performance of the two allocation strategies. The next section shows that healing closes most of the gap to baseline.

5.4 Healing amplifies the advantage

Raw SVD truncation degrades perplexity severely because the model has never seen its own compressed weights during training. *Healing* — a short fine-tuning pass on the compressed model

— allows the surviving parameters to adapt to the new weight configuration and recover much of the lost performance.

We heal both the uniform and entropy-guided compressed models at 80% parameter budget using LoRA [7] (rank 16, $\alpha = 32$, learning rate 2×10^{-4}) for 200 steps on WikiText-2. Table 5 reports the results.

Table 5: Healing amplifies the entropy advantage. At 80% parameter budget, entropy-guided compression starts 5% better before healing and 11% better after healing. The advantage nearly doubles because healing is more effective when the initial compression preserves the most critical parameters. Baseline PPL: 31.83.

Strategy	Compressed PPL	Healed PPL	Improvement
Uniform	3965	209.89	—
Entropy-guided	3751	186.40	11%

After healing, the entropy-guided model achieves PPL 186.40 versus the uniform model's 209.89 — an 11% improvement, up from the 5% advantage before healing. The advantage nearly doubles because healing is more effective when the initial compression preserves the right parameters. Entropy-guided allocation concentrates rank where the spectrum demands it, giving the LoRA adapter a better starting point for recovery.

This result has a practical implication: in any compression pipeline that includes a healing step (which all production pipelines should), the benefit of entropy-guided allocation is *amplified*, not diminished. Spending 45 minutes to compute S_1 before compressing saves far more than 45 minutes of additional healing compute.

6 `entropy-lens`: A Practical Tool

The value of the Entropy-Compression Law depends on how cheaply S_1 can be computed. If measuring the entropy of a 7B model required a GPU cluster, the law would be an interesting theoretical result but not a practical one. We show that it is practical.

6.1 Architecture

`entropy-lens` is a Python package that computes S_1 for every weight matrix of any HuggingFace model. The key design decisions are:

1. **Streaming SVD via safetensors mmap.** Instead of loading the entire model into memory, we memory-map the safetensors files and process one weight matrix at a time. Each matrix is loaded, tensorized, SVD-decomposed at the dominant bipartition, and its spectrum recorded. Peak memory usage is dominated by the single largest weight matrix (typically $d_{\text{model}} \times 4d_{\text{model}}$ for the FFN), not by the total model size.

2. **Architecture-agnostic extraction.** A registry maps HuggingFace model families (GPT-2, LLaMA/Mistral, Qwen, Phi) to their weight matrix naming conventions. Adding a new architecture requires implementing a single `ArchConfig` subclass that specifies the projection names and their shapes.

3. **Dominant-bipartition focus.** Rather than computing the full MPS decomposition (which would require $N-1$ SVDs per matrix), we compute a single SVD at the dominant bipartition — the cut with the largest dimension, which is the compression bottleneck. This reduces the per-matrix cost from $O(N)$ SVDs to one.

6.2 Performance

Table 6 reports the wall-clock time for analyzing three models on an Apple M1 with 16 GB unified memory.

Table 6: `entropy-lens` performance on Apple M1 (16 GB, no GPU). Analyzing a 7B model takes under 45 minutes. The dominant cost is the SVD of the largest weight matrices (4096×14336 for Mistral's FFN layers).

Model	Params	Matrices	Time
GPT-2 Small	124M	72	∼2 min
Mistral-7B	7.2B	224	∼45 min
Qwen2-7B	7.1B	196	∼40 min

6.3 Open-source release

`entropy-lens` is released under the MIT license at https://github.com/nicogarazzo/entropy-lens. The tool provides a command-line interface:

```
entropy-lens analyze mistralai/Mistral-7B-v0.3
```

This produces a CSV of per-matrix S_1, $D_{\min}(\varepsilon)$ at four thresholds, scatter plots of the Entropy-Compression Law fit, and an entanglement heatmap. The results can be fed directly into the rank allocation pipeline.

7 Limitations

We state our limitations directly.

1. **Validated up to 7B, not 70B+.** The largest model tested is 7.2B parameters. Whether the law holds at the 70B–405B scale where compression is most urgently needed remains to be validated. We expect it will — the law *improved* from 124M to 7.2B — but expectation is not evidence. `entropy-lens` can analyze 70B models in principle (via streaming SVD), but we have not yet run this experiment.

2. **Frobenius error $\neq$ perplexity.** The Entropy-Compression Law predicts Frobenius truncation error, not perplexity. As Hsu et al. [6] emphasize, the relationship between per-layer approximation error and downstream task performance is non-trivial. Our compression experiments (section 5) show that the Frobenius-based ranking of layers is informative for perplexity-aware allocation, but we have not derived a formal perplexity-prediction law.

3. **Healing is obligatory for practical compression.** Without post-compression fine-tuning, SVD truncation produces perplexity in the thousands, regardless of allocation strategy. The entropy-guided advantage is in *relative* performance (which strategy degrades less), not in absolute usability of the compressed model without healing. Any practical deployment requires a healing step.

4. **Compression experiments on GPT-2 Small only.** The perplexity comparison between uniform and entropy-guided allocation (section 5) is currently validated only on GPT-2 Small. Mistral-7B compression experiments require GPU cloud infrastructure and are in progress. We include the GPT-2 results because the allocation principle — entropy predicts which layers need more rank — is architecture-independent, even though the specific perplexity numbers are model-specific.

5. **Single bipartition.** We measure S_1 at the dominant bipartition only. A full entanglement characterization would examine all $N-1$ bipartitions and their mutual information structure. For the purpose of compression, the dominant bipartition is the bottleneck, so this is the relevant cut — but the full multi-cut structure may contain additional information for more sophisticated compression schemes.

6. **No comparison against perplexity-aware search methods.** We compare entropy-guided allocation against uniform allocation, not against search-based methods like ARA [1] that optimize ranks by evaluating perplexity at many configurations. Such methods may find better allocations at the cost of orders of magnitude more compute. The entropy-guided approach is intended as a *principled initialization* — a starting point that eliminates most of the search space, not necessarily the final answer.

8 Conclusion

The Entropy-Compression Law scales. On Mistral-7B, the von Neumann entanglement entropy S_1 predicts the minimum bond dimension for a given Frobenius error with $R^2 = 0.997$ — explaining 99.7% of the variance across 224 weight matrices with a single structural measurement per matrix. On Qwen2-7B, the law achieves $R^2 = 0.993$. Both results are stronger than the original GPT-2 validation ($R^2 = 0.865$), because larger models exhibit wider entropy variation across layers, giving the law more leverage.

Beyond prediction, the law is practically useful. Entropy-guided rank allocation outperforms uniform allocation at every compression budget tested, with advantages of 5–38% in perplexity before healing and 11% after healing. As a predictor of layer-wise compressibility, S_1 dominates AlphaPruning's Hill estimator ($R^2 = 0.997$ vs. 0.022); the two metrics are essentially independent ($r = -0.16$), measuring different properties of the singular value spectrum. All computations run on a laptop in under an hour.

One open question is whether the gap between Frobenius prediction and perplexity can be closed — whether a perplexity-aware version of the Entropy-Compression Law exists. Another is whether the law holds at 70B+ scale. But the direction of the evidence is clear: the law gets better with scale, not worse. If the trend continues, the largest models — the ones that need compression most — are the ones where entropy-guided allocation will work best.

A quantum-inspired diagnostic, requiring no labels, no gradients, and no GPU, predicts the compressibility of every layer in a 7B language model with 99.7% accuracy. The singular value spectrum of a trained weight matrix, it turns out, already knows how much you can compress it. You just have to ask it the right question.

Acknowledgments

Computations were performed on Apple Silicon M1 hardware. The author thanks the open-source communities behind PyTorch [11] and HuggingFace Transformers [16].

AI-assisted research statement. This research was conducted by a human author with substantial assistance from large language models (Anthropic's Claude), used for code development, literature review, and manuscript drafting. All experiments were designed, executed, and verified by the author; all numerical results come from the `entropy-lens` code repository, which can be run end-to-end to reproduce them. The author takes full responsibility for every claim.

References

[1] ARA Authors. ARA: Adaptive rank allocation for efficient large language model SVD compression, 2025.

[2] Nicolas Calderon Gonzalez. Entanglement cartography of large language models: An entropy-compression law for tensor network compression, 2026. Preprint. Code: https://github.com/nicogarazzo/entropy-compression-law.

[3] Nicolas Calderon Gonzalez. Why transformers are compressible: From spectral decay to the entropy-compression law, 2026. Preprint. Code: https://github.com/nicogarazzo/entropy-compression-law.

[4] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936. doi: 10.1007/BF02288367.

[5] Jens Eisert, Marcus Cramer, and Martin B. Plenio. Area laws for the entanglement entropy. *Reviews of Modern Physics*, 82(1):277–306, 2010. doi: 10.1103/RevModPhys.82.277.

[6] Yen-Chang Hsu, Ting-Wu Chin, Yilin Shen, and Hongxia Jin. Rethinking truncation-aware decomposition for LLM compression, 2025.

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models, 2021.

[8] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.

[9] Haoran Lu, Yehui Tang, Kai Han, and Yunhe Wang. AlphaPruning: Using heavy-tailed self regularization theory for improved layer-wise pruning of large language models, 2024.

[10] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33 (5):2295–2317, 2011. doi: 10.1137/090752286.

[11] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[12] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://openai.com/research/language-unsupervised.

[13] Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011. doi: 10.1016/j.aop.2010.09.012.

[14] John von Neumann. *Mathematische Grundlagen der Quantenmechanik*. Springer, Berlin, 1932.

[15] Xuan Wang, Shuo Gu, Jiangnan Jia, Hao Zhang, et al. SVD-LLM: Truncation-aware singular value decomposition for large language model compression, 2024.

[16] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pages 38–45, 2020. doi: 10.18653/v1/2020.emnlp-demos.6.

[17] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report, 2024.